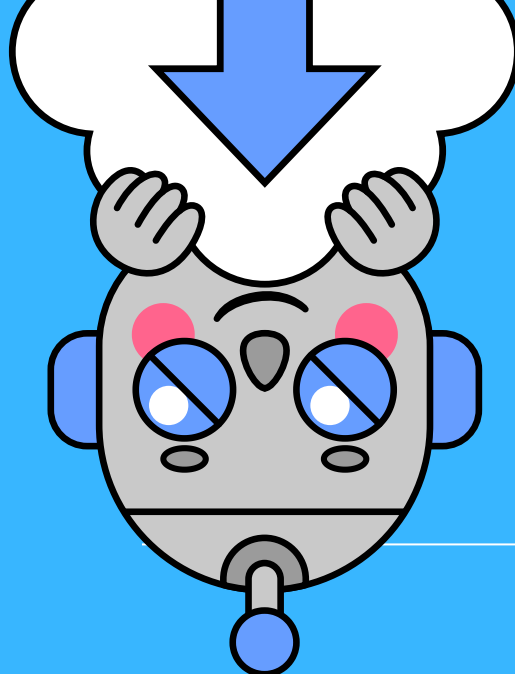


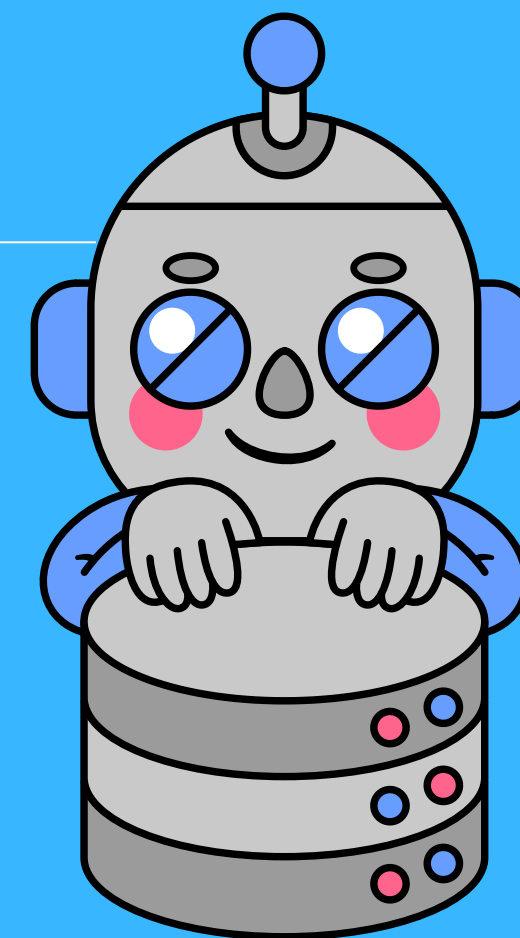
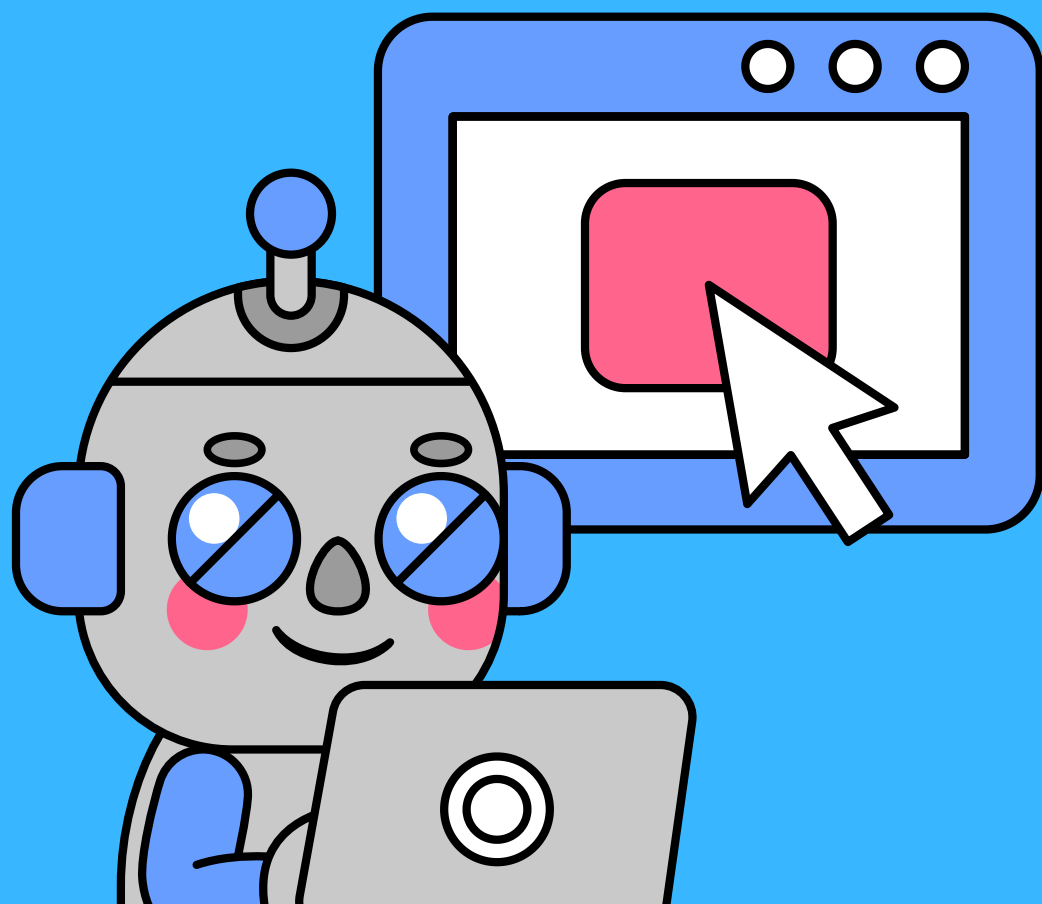


Artificial Intelligence

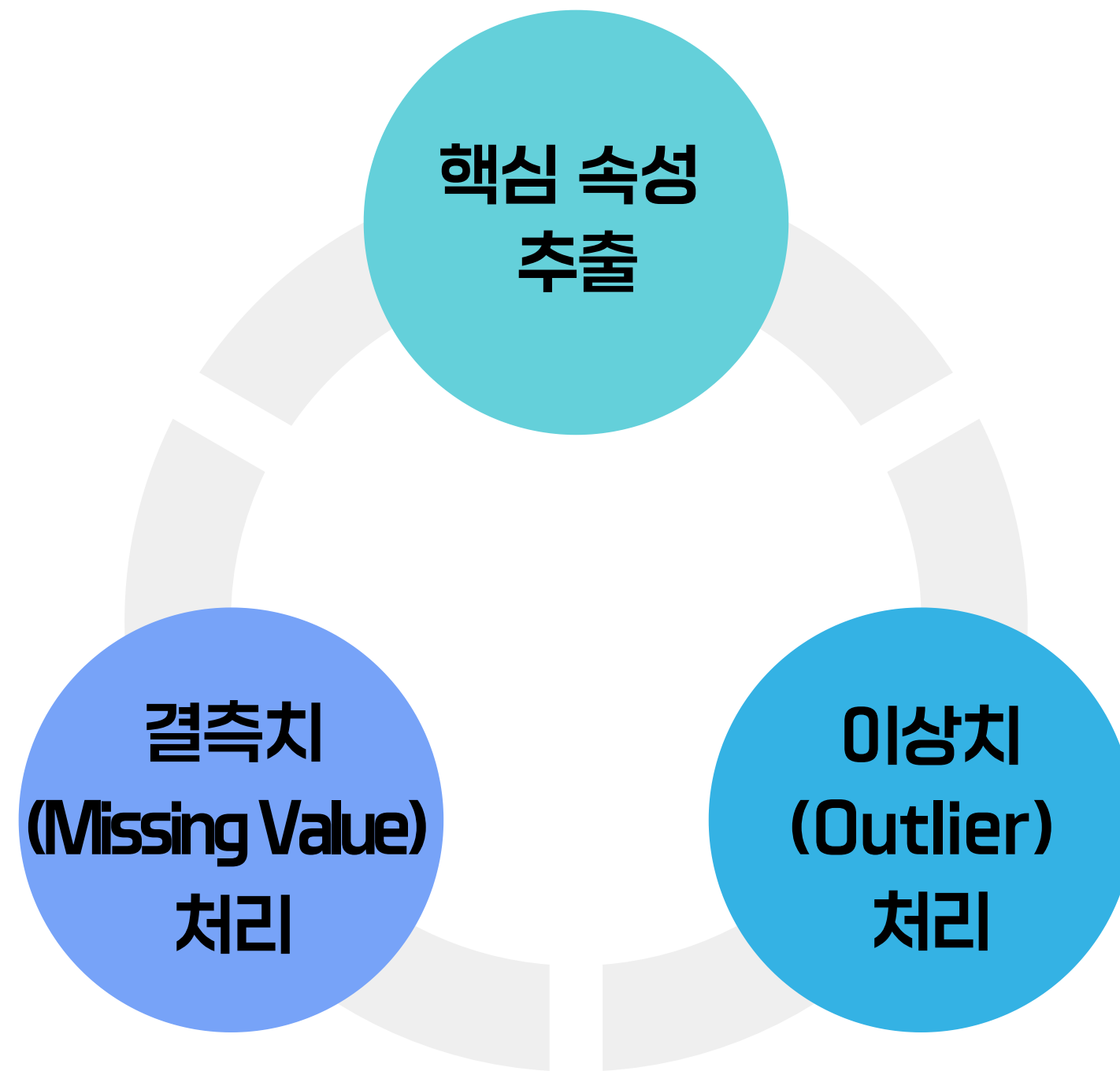
인공지능 기초

인공지능은 어떻게 스스로
똑똑해질까?

Chapter 03. 기계학습과 데이터(2)



1 데이터 전처리



1 데이터 전처리



결측치 (Missing Value)

데이터 수집 과정에서 조사 누락, 시스템 오류, 기기 고장 등으로 인해 값이 채워지지 않고 비어 있는 상태를 의미합니다. 파이썬 Pandas 라이브러리에서는 주로 NaN (Not a Number)으로 표현됩니다.

💡 직관적 비유: 출석부의 공란이나 시험지에서 마킹을 하지 않고 비워둔 '빈칸'과 같습니다.

<중고차 가격 예측 데이터>

연번	모델명	등록지역	주행거리	연식	판매가격 (단위: 만원)
1	아반떼	청주		2022	2100
2	소나타	NaN	15000km	2018	1200
3	코나	대전	50000km	1980	0

2 결측치 처리

탐지 결측치 유무 판단 및 상세 위치 확인

데이터 전처리를 시작할 때, 표 데이터 안에 결측치가 존재하는지 파악하고 어느 변수(열)에 집중되어 있는지 파악해야 합니다. 더 나아가 결측치가 있는 실제 행 데이터를 눈으로 확인하여 누락의 원인이나 패턴을 파악하는 것이 필수적입니다.

> 결측치 탐색 필수 코드 패턴

1. 데이터프레임 전체의 각 칸이 결측치인지 판단 (결측치면 **True**, 정상이면 **False** 반환)

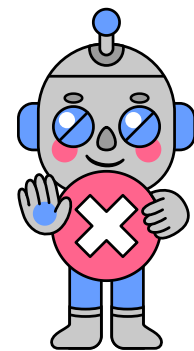
```
df.isna()
```

2. 각 열(Column)별로 존재하는 결측치의 총 개수를 합산하여 요약 출력

```
df.isna().sum()
```

3. 결측치가 하나라도 포함된 행(가로줄) 데이터를 직접 추출하여 눈으로 확인하기

```
df[df.isna().any(axis=1)]
```



잠깐! 헛갈리지 마세요

- `df.isna()` 은 표 전체에 True/False 결과를 뿌려주기 때문에 데이터가 클 때는 한눈에 파악하기 어려움
- 따라서 뒤에 `.sum()`을 붙여 열별 합계로 확인하는 것이 필수적임

2 결측치 처리

처리

결측치 처리 전략 (4대 핵심 알고리즘)

빈칸을 무조건 지우는 것은 정답이 아닙니다. 데이터의 성격에 따라 다르게 대응해야 합니다.

✂ 1. 결측치 삭제 (Drop)

결측치가 포함된 가로줄(행) 전체 또는 세로줄(열)을 데이터셋에서 완전히 배제합니다. 전체 데이터 양이 매우 방대하고 결측치 비중이 낮을 때 안전합니다.

```
# 결측치가 하나라도 포함된 행을 모두 제거  
df.dropna()
```

📊 2. 평균값 대체 (Mean)

해당 변수(열)의 유효한 값들의 산술 평균을 계산하여 빈칸을 일괄적으로 채워 넣습니다. 수치형 데이터 분포가 대칭적일 때 널리 쓰입니다.

```
# 결측치를 해당 열의 평균값으로 채움  
df.fillna(df.mean())
```

⏪ 3. 이전값 대체 (Forward Fill)

결측치가 나타나기 바로 직전(바로 위 행)에 위치한 정상 데이터를 그대로 복사하여 빈칸을 메웁니다. 연속적인 성격을 가진 시계열 데이터 가공에 유리합니다.

```
df.ffill()
```

⏩ 4. 다음값 대체 (Backward Fill)

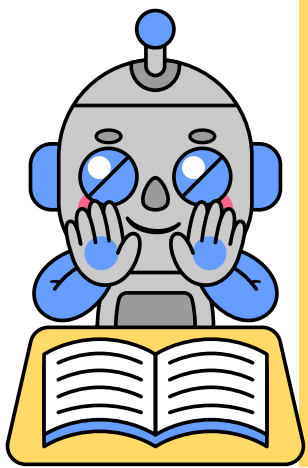
결측치 바로 다음(바로 아래 행)에 등장하는 정상 데이터를 역으로 복사하여 빈칸을 매끄럽게 채워 넣는 가공 방식입니다.

```
df.bfill()
```


2 결측치 처리하기

• 결측치 삭제

df.dropna()



</> 코드 해석

dropna() : na(not a number)를 삭제하라

• dropna()의 두가지 옵션

1) dropna(how="any")

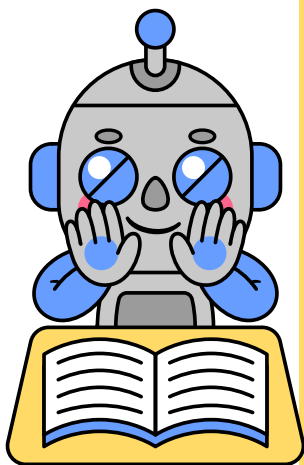
2) dropna(how="all")

	이름	나이	점수
0	짱구	NaN	90
1	철수	5	80
2	유리	5	70
3	NaN	NaN	NaN

2 결측치 처리하기

• 특정 값 넣기

df.fillna(채울 값)



</> 코드 해석

fillna() : na(not a number)에 ()안의 값을 채워라

	이름	나이	점수
0	짱구	NaN	90
1	철수	5	80
2	유리	5	70
3	NaN	NaN	NaN

df[나이].fillna(5) → 나이 속성에 모든 결측치를 5로 채워라

****평균으로 대체하기****

```
a=df[점수].mean()
```

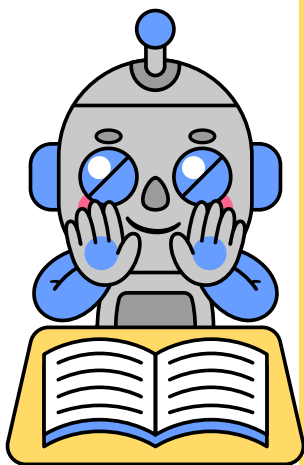
```
df[점수].fillna(a)
```

2 결측치 처리하기

- 이전 값 혹은 다음 값 복제하기

`df.ffill()`

`df.bfill()`



</> 코드 해석

`ffill()`: forward fill (앞의 값, 즉 이전 값으로 채워라)

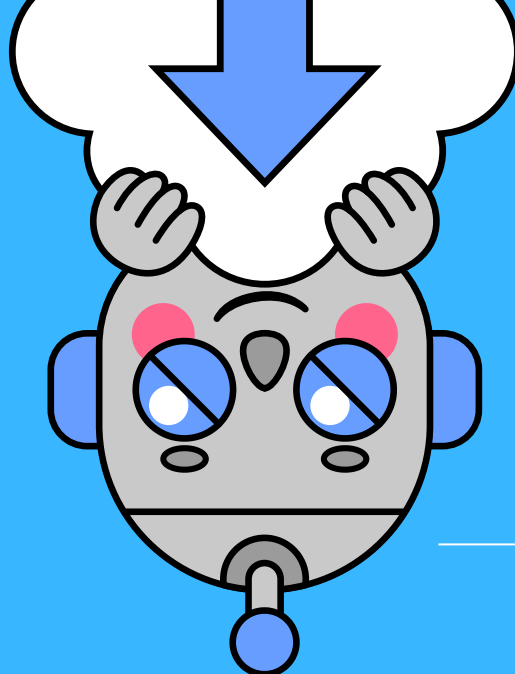
`bfill()`: backward fill (뒤의 값, 즉 다음 값으로 채워라)

	이름	나이	점수
0	짱구	5	90
1	철수	NaN	80
2	유리	5	70
3	NaN	NaN	NaN

`df[나이].ffill()` → 모든 나이가 5로 변경됨

1 데이터 전처리

처리 기법	적용 시점	장점	단점 및 주의점
삭제 (dropna)	데이터양이 아주 많고 빈칸 비율이 낮을 때	왜곡 없는 순수 데이터 보존	중요한 정보 유실 가능성 존재
평균값 대체 (mean)	일반적인 수치형 데이터, 고른 분포	전체 데이터 수 유지, 무난함	데이터의 고유 변동성(분산) 감소
이전/다음값 (ffill/bfill)	주가, 기온, 인구 변화 등 시계열 데이터	시간 흐름과 문맥의 자연스러운 보존	결측치가 장기 연속되면 왜곡 극대화



Artificial Intelligence

NEXT

인공지능과 학습(3) - 이상치 처리

